Ders 9 Genişletilmiş Notu: İşletim Sistemi Yapıları, Süreç Kavramı, Süreç Durumları, PCB ve Zamanlayıcılar

Ders 9 Genişletilmiş Notu: İşletim Sistemi Yapıları, Süreç Kavramı, Süreç Durumları, PCB ve Zamanlayıcılar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. MS-DOS: Basit Yapı ve Komut Yorumlayıcısı
- 2. UNIX'te Komut Çalıştırma: fork, exec ve Çıkış Kodu
- 3. Sistem Programları ve İşletim Sistemi Tasarımı
- 4. İşletim Sistemi Hangi Dille Yazılır?
- 5. MS-DOS'un Katmanları ve BIOS
- 6. Geleneksel UNIX ve Monolitik Çekirdek
- 7. Katmanlı Yaklaşım
- 8. Mikroçekirdek
- 9. Yüklenebilir Çekirdek Modülleri
- 10. Karma Yapılar
- 11. Bölüm 3'e Giriş: Süreçler
- 12. Süreç Kavramı
- 13. Bellekte Süreç
- 14. Süreç Durumları
- 15. Süreç Kontrol Bloğu (PCB)
- 16. Bağlam Değişimi
- 17. İş Parçacıklarına Kısa Giriş
- 18. Süreç Zamanlama Kuyrukları
- 19. Kısa, Uzun ve Orta Vadeli Zamanlayıcılar

1. MS-DOS: Basit Yapı ve Komut Yorumlayıcısı

Ders, Bölüm 2'de kalınan yerden, slayt 2.26 civarından devam etti. Konu, gerçek işletim sistemi örnekleri üzerinden işletim sistemi yapılarıydı. İlk ve en basit örnek MS-DOS'tur. Hoca "MS" kısmına takılmamamızı söyledi: Microsoft'tan önce de DOS (Disk Operating System) vardı.

MS-DOS oldukça basite indirgenmiş (simplistik) bir yapıdır. Hocaya göre burada süreç (process) kavramından söz etmek bile zordur. Çünkü sistem tek görevlidir (single tasking): herhangi bir anda yalnızca bir program çalışır, başka bir şey olmaz. Bu yüzden şu sorunların hiçbiri yoktur:

- programlar arasında geçiş yapmak,
- bellekte birden fazla süreç tutmak,
- süreçleri bellekten alıp geri koyarken oluşan boşluklarla uğraşmak,
- genel olarak bellek yönetimi (memory management).

Sistemde bir çekirdek (kernel) ve bu çekirdeğe komut gönderdiğiniz bir komut yorumlayıcısı (command interpreter) vardır. Eski zamanlarda bilgisayar operatörünün yaptığı işler artık bu

komut yorumlayıcısı üzerinden yapılır: bir programı yüklersiniz, çalıştırırsınız, hata verirse görürsünüz, izlersiniz. Sistem bundan ibarettir.

1.1 Slayttaki İki Resim

Slaytta MS-DOS'un bellek düzeni iki resimle gösteriliyordu: solda sistem açılışı, sağda bir program çalışırken. Hoca iki resim arasındaki farkları sordu. Birinci fark açıktır: sağdaki resimde boş bellek (free memory) daha azdır, çünkü bir program çalışmaya başlamıştır.

İkinci farkı bir öğrenci buldu: komut yorumlayıcısı solda daha büyük, sağda daha küçük çizilmiştir. Hoca bunun bir çizim hatası olmadığını, bilerek böyle çizildiğini vurguladı.

Nedeni şudur. Sistemde bellek yönetimi yoktur ve herhangi bir anda tek program çalışır. Bu yüzden çalıştırabileceğiniz programın boyutu, boş belleğiniz kadardır. Bu boyutu olabildiğince büyük tutmak için komut yorumlayıcısı iki parçaya ayrılmıştır:

- **Yerleşik (resident) kısım:** Her zaman bellekte durur.
- **Geçici (transient) kısım:** Bir program çalıştırılacağı zaman bellekten kaldırılır ve yerini programa bırakır.

Program bitince komut yorumlayıcısına geri dönülür. Bu sırada kalıcı (yerleşik) kısım, geçici kısmı diskten yeniden belleğe kopyalar. Böylece çalışan program için daha fazla boş bellek oluşturulmuş olur. Bütün mantık bundan ibarettir.

> **Ders sonrası not:** Hoca 00:08:30 civarında, program çalışırken "resident kısım" bellekten kaldırılıyor dedi (altyazıda "resilient"). Hemen ardından anlattığı akış doğrudur: bellekten kaldırılan **geçici (transient)** kısımdır, program bitince **yerleşik (resident)** kısım geçici kısmı geri yükler.

2. UNIX'te Komut Çalıştırma: fork, exec ve Çıkış Kodu

Daha karmaşık örnek olarak hoca FreeBSD'yi gösterdi. FreeBSD bir Unix türevidir. Unix ailesinin bir kolu AT&T laboratuvarlarından, diğer kolu Berkeley Software Distribution (BSD) olarak devam eder.

Bu sistem çok görevli (multitasking) bir ortamdır. Farklı kullanıcılar ve birden fazla süreç vardır, süreçler arasında zaman paylaşımı (time sharing) yapılır. Burada artık gerçek anlamda süreçlerden söz ederiz.

Burada da bir çekirdek ve komut girmenizi sağlayan bir yorumlayıcı vardır. Yorumlayıcının işi, girdiğiniz komutlara karşılık programları çalıştırmaktır. Bunu süreç kavramıyla yapar:

1. Önce `fork` sistem çağrısıyla yeni bir süreç yaratılır.
2. Sonra `exec` sistem çağrısıyla çalıştırmak istediğiniz program bu sürecin içine yüklenir.

Unix'lerde bir teamül (gelenek) vardır. Her süreç tamamlandığında bir sonuç üretir. Bu sonuç bazen birden fazla bilgi içeren bir geri dönüş olabilir, bazen de yalnızca "başarılı / başarısız" gibi tek bir bilgi olabilir. Genel yaklaşım şudur:

- Süreç tamamlandığında her şey normalse, dönüş kodu (return code) olarak **0** döndürür.
- Bir hata ya da problemle karşılaşıldıysa **0'dan farklı** bir değer döndürür.

Kontroller hep bunun üzerinden yapılır. Kabuk (shell) gibi komut çalıştırdığınız yerlerde, çalıştırılan komutun başarılı olup olmadığını anlamak için hep bu geri dönüşe bakılır.

Hoca bunun süreç yönetiminde (process management) tekrar konuşulacağını söyledi. Süreç durumları konuşulurken, tamamlanmış bir sürecin bile ayrı bir durumda tutulması gerektiği görülecekti. Çünkü bu çıkış bilgisinin, süreci yaratan süreç tarafından kullanılması istenir. Bu konu aynı derste, Bölüm 14'te işlendi.

3. Sistem Programları ve İşletim Sistemi Tasarımı

Hoca sistem programlarını farklı biçimlerde sınıflandırabileceğimizi, slayttaki sınıflandırmaya bağlı kalmak zorunda olmadığımızı söyledi. Teknik olarak bir işletim sistemi, üzerinde çalışacak programlar için şunları sağlamalıdır:

- programları oluşturabilmek, yani geliştirmeye (development) imkân vermek: derleyici (compiler), çevirici (assembler), hata ayıklayıcı (debugger) gibi araçlar,
- oluşturulan programları yükleyip çalıştırabilmek,
- haberleşme sağlamak,
- dosya yönetimi,
- durum bilgisi (status information): sistem saati, bağlı bir cihazın durumu, sistemin genel kaydı (log) gibi bilgiler.

Bunların hepsi sistemin bize sağladığı imkânlardır ve hepsi sistem programıdır.

Bu sistem programlarının işletim sistemiyle nasıl bütünleştirileceği, nereye yerleştirileceği, nasıl bir yapı içinde kalacağı, işletim sisteminin **mimarisini** belirler. Hoca burada ilk derslerdeki "tarayıcı (browser) işletim sisteminin bir parçası mıdır?" tartışmasını hatırlattı. Her parça, genel mimari açısından farklı şekillerde gerçeklenebilir.

Hocaya göre işletim sisteminin tasarımı ve gerçeklenmesi matematiksel bir problem değildir. Bir denklem kurup çözebileceğiniz bir problem değildir. Bilinen bazı temel yaklaşımlar vardır ve bunlar üzerinden ne yaparsanız ne olacağını ortaya koyabilirsiniz.

İşletim sistemi geliştirmek bir süreçtir. Özellikle uzun süredir var olan ve gelişmeye devam eden işletim sistemlerinde hep bir iyiye gidiş görülür. Hoca bunun Windows için bile geçerli olduğunu söyledi. Bu yüzden bir işletim sisteminin ne kadar uzun yaşadığı, yapısının ve servislerinin oturması açısından önemlidir.

Bir öğrenci "iyiye gidiş" ile neyin kastedildiğini sordu. Hoca şöyle açıkladı: İlk Unix mimarisi monolitiktir, önünüze kütük gibi konan koskocaman bir yapıdır. Zamanla bunu değiştirmek için adımlar atıldı: işi "makro çekirdekten mikroçekirdeğe" doğru çevirelim, bunu katmanlarla yapalım, katmanlar olmadı modüllerle yapalım gibi bir değişiklik silsilesi yaşandı. Eski Unix'lere göre bugün kullandığımız Unix tabanlı sistemlerde iyiye doğru bir fark vardır. Aynı şey Windows için de geçerlidir, sadece oradaki değişim daha yavaş ve daha karmaşıktır.

4. İşletim Sistemi Hangi Dille Yazılır?

İlk işletim sistemlerinde çoğunlukla Assembly ile yazılmış yapılar görülür. Her şey Assembly üzerinden geçer. O dönemde alternatif olarak biraz daha üst seviyede ALGOL ve PL/I gibi diller de kullanılmıştır.

Unix'i ortaya çıkaran ekip ile C programlama dilini ortaya çıkaran ekip aynıdır. C'nin oluşturulma amacı, Assembly'ye göre daha esnek ve daha taşınabilir (portable) bir dil ortaya koymak ve bununla işletim sistemi yazmaktır.

Bugün birçok işletim sistemi C ve C++ ile yazılmıştır. En alt kısımda, önyükleyici (bootloader) kısmında ve çekirdeğin belli yerlerinde hâlâ Assembly görülür, ama genel yapı C ile yazılır. Sistem programları da genellikle C ya da C++ iledir. Bunun dışında Perl, Python ve kabuk betikleri (shell script) ile yazılmış birçok kısım da vardır. Hoca, Python'un kendisinin de C ile yazıldığını hatırlattı.

Temel ölçüt şudur: Programlama dillerinde yukarı çıktıkça, yani donanımdan uzaklaştıkça araya farklı katmanlar sokmuş olursunuz.

- **Artısı:** Donanımdan soyutlama (hardware abstraction) sağlarsınız ve taşınabilir olursunuz. A sisteminden B sistemine çok daha hızlı geçebilirsiniz.
- **Eksisi:** Yavaşlama.

  Ders sonrası not: Altyazıda "SMB'li" olarak geçen dil Assembly'dir; "Algor, PL1" ise ALGOL ve PL/I'dır.

5. MS-DOS'un Katmanları ve BIOS

Hoca bundan sonra yapıları adım adım inceleyeceklerini söyledi: MS-DOS, Unix, katmanlı yapı, modül yapısı ve mikroçekirdek. Zaman kalırsa Windows'a da bakılacaktı.

MS-DOS'un işlevselliği sınırlıdır. Herhangi bir modül yapısı ve çok görevlilik (multitasking) yoktur. Yapı aşağıdan yukarıya şöyledir:

- En altta donanım ve donanıma gömülü yazılım (firmware), yani BIOS vardır. BIOS doğrudan cihazlara erişmek için kullanılır.
- Üstünde DOS işletim sistemi ve onun içinde DOS aygıt sürücüleri (device drivers) vardır.
- En üstte yerleşik sistem programları (resident system programs) ve uygulamalar vardır.

Dikkat edilmesi gereken nokta, katmanlar arasında sıkı bir ayrım olmamasıdır. Yerleşik sistem programı isterse MS-DOS aygıt sürücüleri üzerinden donanıma ulaşır, isterse doğrudan BIOS'taki sürücüleri kullanır. Bilgisayarla birlikte gelen BIOS, birçok cihaz için temel sürücülere sahiptir; örneğin diske okuma-yazma için sorun yoktur. Slayttaki grafiğe göre bir uygulama bile, biraz uğraşarak ("kan, ter, gözyaşı dökerek") kendi içinden doğrudan BIOS sürücüsünü kullanıp bir cihaza erişebilir.

Fakat gelen her cihaz BIOS tarafından desteklenmeyebilir. Hocanın örneği CD-ROM sürücüleriydi. CD-ROM'ların geometrik yapısı ve erişim biçimi sabit disklerden farklı olduğu için BIOS sürücüsü doğrudan kullanılamıyordu. MS-DOS'un kendi sürücüsü vardı. Bu sürücü kesme güdümlü (interrupt driven) değildi, yoklama (polling) ile çalışıyordu.

Normalde şunu beklersiniz: CD'yi takıp kapağı kapattığınızda bir kesme oluşur ve sistem durumu öğrenir. Ama yapı öyle değildi. Sürücü belli zaman aralıklarında gidip "orada bir şey var mı?" diye kontrol ediyordu. Bir şey okunacağı zaman da yine "okundu mu, okundu mu?" diye yoklayarak bekliyordu.

6. Geleneksel UNIX ve Monolitik Çekirdek

Unix'e geçince, ilk Unix'te bile yapı karmaşıktır. Çünkü:

- birden fazla kullanıcı (multiple user) desteklenir,
- her kullanıcının birden fazla programı, dolayısıyla birden fazla süreci olabilir,
- bunlar arasında zaman paylaşımı yapılır,
- bu da beraberinde bellek yönetimini getirir.

Yapılacak iş bol olduğu için çekirdek de büyüktür. İlk Unix sürümleri iki parçaya ayrılabilir: **sistem programları** ve **çekirdek**.

Slayttaki resimde en üstte kullanıcılar vardır. Altında kabuk ve komutlar, derleyiciler ve yorumlayıcılar, sistem kütüphaneleri gibi sistem programları bulunur. Bunlarla çekirdek arasında boydan boya uzanan sistem çağrısı arayüzü (system-call interface) vardır; çünkü işletim sistemine bu arayüz üzerinden erişilir.

Çekirdek tarafında şunlar vardır:

- sinyaller (signals),
- G/Ç alt sistemi (I/O subsystem),
- terminal sürücüleri, teyp ve disk sürücüleri, yani bütün sürücüler,
- dosya sistemi (file system),
- CPU zamanlaması (CPU scheduling),
- bellek yönetimiyle ilgili sayfa değiştirme (page replacement), istem üzerine sayfalama (demand paging) ve sanal bellek (virtual memory).

Kalabalık bir yapı olduğu için bu mimaride çekirdeğin büyük olması gerekir. Buna monolitik yapı denir.

6.1 Büyük Çekirdeğin Sakıncaları

Hoca büyük çekirdeğin dezavantajını sordu. Öğrencilerden gelen cevaplardan biri "C yerine Python kullanmak gibi, bellek kullanımı fazla olabilir" idi. Hoca bu benzetmeyi biraz tuhaf buldu ama diğer cevapların birçok noktada doğru olduğunu söyledi.

Hocanın topladığı sakıncalar şunlardır:

- **Bellek:** Çekirdek büyüdükçe ihtiyaç duyduğu bellek alanı artar. Oysa işletim sistemi tasarımında, işletim sistemi ne kadar az sistem kaynağı kullanırsa o kadar iyidir.
- **Hız:** Böyle bir çekirdeği yüklemek, çalıştırmak ve o yüklüyken diğer programları çalıştırmak yük hâline gelir. Kabaca söylersek sistemi yavaşlatır.

Hoca hız konusunu zamanlama (scheduling) üzerinden açıkladı. Zamanlama önemlidir, çünkü sistemin ne kadar etkin çalıştığını ve kaynağın ne kadar iyi paylaştırıldığını belirler. Fakat zamanlama kararı çok kısa sürede verilmelidir. Aksi hâlde sistem kaynağını gerçekten yapmak istediğiniz işe değil, o işi kimin yapacağına karar vermeye harcarsınız.

Bu sakıncalar yüzünden "o zaman çekirdeği küçültelim" denmiştir. Sonraki yapıların hepsi bu fikirden çıkar.

7. Katmanlı Yaklaşım

Katmanlı yaklaşımda (layered approach) işletim sistemi katmanlara bölünür. Slaytta ortada katman 0 donanımdır (hardware). Donanımın hemen üstünde işletim sisteminin çekirdek kısmı yer alır. Bunun üstünde birden fazla katman vardır.

Kural şudur: Her katman yalnızca alt katmandan servis alır ve üst katmana servis verir. Üst katman doğrudan herhangi bir şeye erişmez. İşler ve işlevler tanımlanır, katmanlar birbirinden destek alarak ilerler. En üst katman yalnızca veri oluşturmakla ilgilenir; alta inene kadar her katman kendi yapması gereken işi yapar.

Böylece donanımın hemen üstündeki katmanda kalan işletim sistemi çekirdeği küçültülür, diğer işler üst katmanlarda kalır. Hocaya göre bu, belli açılardan güzel bir yaklaşımdır ve amaç çekirdeği küçültmektir.

Bu yapıda her katman üst katmana servis vereceği için katmanlar arasında haberleşme gerekir. Aynı katmandaki servislerin kendi aralarında haberleşmesi de gerekir.

8. Mikroçekirdek

Hoca bu yapının örneği olarak Mach çekirdeğini verdi ve bunun bir mikroçekirdek (microkernel) olduğunu söyledi. Mac OS X ile bu yapıya geçildiğini, fakat güncel sürümlerin tam olarak böyle olmadığını, daha sonra modüllere ve belli açılardan karma (hybrid) yapıya geçildiğini anlattı.

Sonraki slaytta yapı daha net görülür. Katmanlı yapıdaki gibi ortada donanım (katman 0) vardır. Üstünde mikroçekirdek yer alır ve katman 1'de yalnızca o vardır. Mikroçekirdek yalnızca en temel servisleri verir:

- bellek yönetimi (memory management),
- CPU zamanlaması (CPU scheduling),
- süreçler arası iletişim (IPC, Inter-Process Communication).

Çekirdeğin kendisi budur. Bunun üstündeki her şey, örneğin dosya sistemi ve aygıt sürücüleri, kullanıcı modunda (user mode) çalışır.

Bu servislerin birbirleriyle konuşabilmesi için bir mesajlaşma yapısı gerekir ve bu mesajlaşma çekirdek üzerinden yapılır. Örneğin:

- Bir uygulama programı dosya sistemiyle konuşacaksa bir mesaj üretir. Bu mesaj mikroçekirdek üzerinden dosya sistemine gelir.
- Dosya sistemi bir diskten bir şey okuyacak ya da yazacaksa aygıt sürücüsüyle konuşur. Bu mesaj da yine mikroçekirdek üzerinden geçer.

8.1 Artısı: Yalıtım

Hocaya göre bu, belli açılardan çok güzeldir. Dosya sistemini kullanıcı modunda gerçeklemiş olursunuz. Bu yüzden her kullanıcının dosya sistemini birbirinden tamamen yalıtabilirsiniz. Bugün kullandığımız konteyner (container) teknolojisinde bu fikirden yararlanılır. Konteyner teknolojisiyle birleştirildiğinde korumalı (protected) bir yapı da sağlanabilir.

Bir öğrenci konteynerlerle ilgili bir soru sordu. Hoca bunun şu anın konusu olmadığını, dağıtık sistemler (distributed systems), konteynerler ve sanallaştırma (virtualization) konularına gelindiğinde ayrıca konuşulacağını söyledi.

8.2 Eksisi: Mesajlaşma Maliyeti

Tek bir makinede çalışmaya döndüğümüzde şöyle bir sıkıntı vardır: Her mesajlaşma, kullanıcı modundan çekirdek moduna, sonra da çekirdek modundan kullanıcı moduna bir geçiş demektir. Sürekli mesaj aktarmak için sürekli kesme (interrupt) oluşturmanız gerekir; hocanın ifadesiyle bir mesajı aktarmak için iki kesme gerekir. Haberleşmenin getirdiği bu ekstra yük performansı düşürür.

> **Ders sonrası not:** Derste katmanlı yaklaşım ile mikroçekirdek aynı anlatım içinde geçti ve Mach, katmanlı yapının örneği gibi sunuldu. Bunlar büyük olasılıkla ayrı tutulması gereken iki yaklaşımdır: katmanlı yaklaşımda işletim sistemi katmanlara bölünür; mikroçekirdekte ise çekirdek en aza indirilir ve servisler kullanıcı modunda süreçler olarak çalışır. Mach bir mikroçekirdektir. Ayrıca altyazıdaki "Mac kernel" ifadesi Mach çekirdeğidir.

9. Yüklenebilir Çekirdek Modülleri

Hoca, katmanlı ve mikroçekirdek yapının haberleşmedeki geçişler ve mesaj aktarımı yüzünden performansı düşürdüğünü özetledi. Buna alternatif olarak üretilen yaklaşımlardan biri modül kavramıdır: yüklenebilir çekirdek modülleri (loadable kernel modules). Bugün kullandığımız Linux sistemleri genel olarak bunu kullanır.

Çekirdeğin herhangi bir parçasını doğrudan çekirdeğe gömülü hâle getirebilirsiniz ya da bir modül olarak gerçekleyebilirsiniz. Modül, bir modül arayüzü üzerinden çekirdekle haberleşir.

Güzel tarafı şudur: Yapabildiğiniz her şeyi çekirdek modülü olarak yaparsanız, bir parça ihtiyacınız olduğu sürece bellekte olur, çalışır ve çekirdeğin bir parçası olur. İhtiyacınız bittiğinde kaldırılır. Linux'ta:

- İhtiyacınız olduğunda `insmod` ile örneğin bir aygıt sürücüsünü yüklersiniz.
- `modprobe` kullanırsanız, o aygıt sürücüsü başka modüllere ihtiyaç duyuyorsa onlar da otomatik yüklenir.
- İhtiyacınız bittiğinde `rmmod` ile kaldırırsınız.

Derlenmiş bir modül sistemde bulunuyorsa, çalışma zamanında (runtime) "modülü yükle" deyip onu sistemin bir parçası hâline getirebilirsiniz.

Bu yaklaşım Linux'ta, Solaris'te, hatta artık karma bir yapıda olan Mac OS'ta da vardır. Windows'ta bile belli açılardan vardır. Kitaptaki örnek Solaris'tir: ortada bir çekirdek vardır ve bu çekirdek farklı işleri yapabilmek için modüllere bölünmüştür. Örneğin bir dosya sistemine erişeceksiniz; o dosya sistemiyle ilgili bir çekirdek modülü vardır.

10. Karma Yapılar

Hoca, tanımlanan yapıların (katmanlar, modüller, mikroçekirdek) güncel bir işletim sisteminde tek başına yer almadığını vurguladı. Farklı işletim sistemlerinde bunlar farklı şekillerde bir araya getirilmiştir:

- **Linux:** Çekirdeğin büyük ölçüde modüler olduğunu söyleyebiliriz, ama monolitik bir kısmı da vardır.
- **Windows:** Çok büyük bir oranı monolitiktir; her şey gömülü, kocaman bir parçadır. Fakat bazı sistem parçaları için kenarda çalışan, mikroçekirdeğe benzeyen bir kısım da vardır; küçültülmüş modüllerin eklenip çıkarılabildiği bir yapı vardır. Hoca, Windows 10'un mimarisini çizmenin karmaşık olacağını, çünkü bir kısmının monolitik, bir kısmının "mikroçekirdek olduğu iddia edilen" bir yapı olduğunu, arada da modüller bulunduğunu söyledi. Hocanın tahminine göre ilerleyen zamanda bu kısım, söz konusu iyileşme süreci içinde artacaktır.
- **Mac OS X:** Modüler ve katmanlı bir yaklaşım vardır. Bir tarafta BSD çalışma ortamını içeren ve ona destek veren bir çekirdek parçası ve modüller vardır; öbür tarafta katmanlar görülür. Hoca, slayttaki resmin eski sürüme ait olduğunu, güncel sürümün iyice modüler hâle geldiğini söyledi.

  Ek bilgi (derste anlatılmadı): Mac OS X'in çekirdeği XNU, büyük olasılıkla baştan beri Mach mikroçekirdeği ile BSD kodunu aynı çekirdek adres alanında birleştiren

karma bir çekirdektir. Bu yüzden “önce saf mikroçekirdeğe geçildi, sonra modüle dönüldü” diye okunmamalıdır. Aygıt sürücüleri için ayrıca yüklenebilir çekirdek uzantıları (kernel extensions) kullanılır.

10.1 Bölüm 2’nin Geri Kalanı

Hoca slayt 2.45-2.46 civarında durdu. Bölüm 2’nin bundan sonrası üzerinde zaman harcamak istemediğini, ama bu kısımdan sorumlu olunduğunu söyledi. Öğrencilerden kitaptan okumalarını ve slaytlardan takip etmelerini bekledi. Ardından Bölüm 3’e geçildi.

11. Bölüm 3’e Giriş: Süreçler

Bölüm 3’te hep süreç (process) üzerine konuşulacaktır. Ele alınacak konular:

- Süreç kavramı nedir?
- Süreç zamanlaması (process scheduling) nasıl yapılır? Aslında sürecin yaşam döngüsü nedir? Bir süreç bir yerden sisteme dahil olur, sistemde bir süre kalır ve çıkar. Bunun süreç bazında nasıl olduğuna bakılacaktır.
- Süreçler üzerindeki işlemler: En basitinden süreç başlatabilir ve sonlandırabilirsiniz. Bunun dışında işletim sisteminin arka planda süreçler üzerinde yaptığı başka işlemler de vardır.
- Süreçler arası iletişim (IPC) ve örnek sistemler: FIFO’lar, borular (pipes), sinyaller, mesaj kuyrukları (message queues) ve paylaşılan bellek (shared memory).
- İstemci-sunucu (client-server) mimarileri: Burada daha çok genel kültür düzeyinde kalınacaktır. Asıl yeri ağ teknolojileri dersleri ve seçmeli ileri ağ programlama dersleridir.

Hoca, burada konuşulan teorik bilginin uygulamalarda pratiğe döndürüleceğini, uygulamada neyin nasıl yapıldığının ayrıca gösterileceğini söyledi.

11.1 Job, Task ve Process

Hoca, kitabı okurken ve kendisini dinlerken “job”, “task” ve “process” kavramlarının birbirinin yerine kullanıldığını göreceğimizi söyledi. Kitapta da bunlar aynı anlama gelecek şekilde geçişli kullanılır.

Bunun nedeni, işletim sisteminin bir günde olup bitmiş bir yapı olmamasıdır. İhtiyaç ilk ortaya çıktığında sistemler “job” gibi kavramlarla tarif edildi. Sonra çok görevlilikle birlikte “task” kavramı ortaya atıldı. Bugün ise süreç (process) ve iş parçacığı (thread) konuşuyoruz. Sürece bakınca bunların hepsi aynı kavrama gelir; sadece değişen sistemle birlikte kabuk değiştirmişlerdir. Hoca derste çoğunlukla “process” terimini kullanacağını söyledi.

12. Süreç Kavramı

En basit tanımıyla süreç, **programın çalışan hâlidir**.

Hoca bunu nesne yönelimli programlamadaki (Object Oriented Programming) sınıf (class) ile nesne (object) arasındaki ilişkiye benzetti. Program, belli bir mimaride belli bir mantığa göre çalışması beklenen, art arda dizilmiş komutlar (instructions) bütünüdür. Bunları bir araya getirip ikincil belleğe (diske) koyarsınız; orada program olarak durur. Program belleğe alınıp bu akışa göre sırayla çalıştırıldığında süreç olur. Arada komutlardan biri bir atlama (jump) ise oraya atlanır, ama çalışma yine o bütün içinde sürer.

Bir sürecin çalışması için nelere ihtiyaç vardır?

- **Kod kısmı:** Buna genellikle text bölümü (text section) denir.
- **Program sayacı (program counter):** Kod işlemcide çalışır ve çalıştığı sürece bir sonraki adımı gösteren bir program sayacı vardır. Bu, çok görevli ortamda çok önemlidir. Çünkü bir süreci biraz çalıştırıp kenara alacak, başkasını çalıştıracak, sonra ilkine geri döneceğiz. Nerede kaldığımızı takip edebilmeliyiz.
- **Yazmaçlar (registers):** İşlemcide çalışan bir süreç mutlaka sonuç üretir ve bu sonuçları yazmaçlar üzerinde üretir. Örneğin bir döngüdesiniz ve döngüde nerede kaldığınızı CX yazmacından biliyorsunuz. Süreci kenara koyup geri yüklediğinizde bu yazmacın değeri

farklıysa çalışma tuhaf bir hâle gelir: yanlış çalışır, eksik çalışır, istemediğiniz bir sonuç üretir. Bu yüzden yazmaç kümesi (register set) de bir yerde saklanmalıdır.
- **Erişilen kaynaklar:** Programın hangi dosyalara ve hangi cihazlara eriştiği de bu işin içindedir.
- **Data, stack ve heap bölümleri:** Bir sonraki başlıkta ayrıntılı anlatılmıştır.

13. Bellekte Süreç

Programın içinde bir data bölümü, bir stack bölümü ve bir heap bölümü vardır:

- **Data bölümü (data section):** Programın genel (global) değişken olarak tanımladığı bilgi burada tutulur.
- **Stack:** Geçici bilgi burada tutulur. Örneğin bir fonksiyon çağıracaksınız ve parametre geçireceksiniz; parametreleri stack'e koyarsınız. Fonksiyon bitince geri dönülecek adresi de stack'e koyarsınız. İş bitince bunu çektiğinizde (pop) fonksiyonu çağırdığınız yere geri dönmüş olursunuz.
- **Heap:** Dinamik olarak oluşturduğumuz alandır. Süreç kendi dünyasında çalışır: stack'e fonksiyon bilgisi basar, data alanındaki veriler üzerinde işlem yapar. Daha fazla belleğe ihtiyaç duyduğunda dinamik bellek ayırma (dynamic memory allocation) yapar ve ona bir heap alanı açılır.

Hoca heap'in büyümesini şöyle anlattı: Her ayırmada heap için ayrılan segment boyutu aşıldığında bir segment daha açılır.

> **Ek bilgi (derste anlatılmadı):** Unix benzeri sistemlerde heap, büyük olasılıkla bellek ayırıcının (`malloc` gibi) çekirdekten istemesiyle büyütülür: küçük ayırmalarda çoğunlukla `brk`/`sbrk` çağrılarıyla heap'in sınırı yukarı kaydırılır, büyük ayırmalarda ise `mmap` ile ayrı bir bellek bölgesi alınır. Dersteki "yeni segment açılır" anlatımı bunun basitleştirilmiş hâlidir. Ayrıntılar bellek yönetimi bölümünde ele alınır.

13.1 Bellek Resmi

Bu yapıda boyutu değişebilen iki yer vardır: stack ve heap. Text (kod) ve global data ise sabittir. Program nasıl başlatılırsa başlatılsın (çift tıklayarak, dokunarak ya da komut satırından), belleğe kopyalandığı anda kod ve data kısımları kopyalanır.

Heap ve stack'in boyutu değişebileceği için bellekte biri aşağıdan yukarı, diğeri yukarıdan aşağı genişleyecek şekilde yerleştirilir:

- Stack en üstten aşağı doğru adreslenir.
- Heap, data bölümünün hemen üstünden başlayıp yukarı doğru adreslenir.

Hoca resimde bir şeye daha dikkat çekti: Grafiğin sol tarafında altta **0**, üstte **max** yazar. Yani sistem ne kadar bellek destekliyorsa, bellek boyutu ne olabilecekse tamamı görünür. Sanki belleğin tamamı bu sürece aittir.

Aslında öyle değildir. Örneğin 4 GB bellek varsa, 4 GB'ın tamamı o süreç tarafından kullanılmaz. En basitinden arkada işletim sisteminin kendisi ve başka süreçler vardır. Ama sistem olarak her sürece, sanki işletim sistemindeki tek süreç kendisiymiş gibi davranırız: "bütün bu alan senin, istediğin gibi kullan" deriz.

Bunu yapabilmek için arkada yalnızca sürecin ayırdığı kadar segment oluşturulur ve bunlar belleğe yerleştirilir. Süreçten beklenen, tek bir alan içinde bir adresten başlayıp adım adım ilerlemesi, arada atlama varsa onu yapmasıdır. Bu adresleme tamamen mantıksal (logical) olarak yapılır. En alt seviyede ise bunlar birbirinden bağımsız, parça parça segmentlerdir. Bunun amacı belleği daha etkin kullanmaktır. Hoca bunun bellek yönetimi konusunda ayrıca konuşulacağını söyledi.

14. Süreç Durumları

Her süreç yaşam döngüsü boyunca bir grup farklı durumdan (state) geçer. Farklı işletim sistemlerinde, sistemin yapısına ve özelliğine bağlı olarak farklı durumlar görülebilir. Hoca gerçek zamanlı (real-time) olmayı hedefleyen bir işletim sisteminin durum diyagramının daha

farklı olduğunu söyledi. Linux ya da Windows'un süreç durum diyagramlarında da küçük farklar görülebilir.

Temelde bir sürecin yaşam döngüsünü göstermek için beş durum içeren bir diyagram kullanılır:

Durum	Anlamı
new	Sürecin yaratılma anı
ready	Süreç hazırdır, işlemcide çalıştırılabilir hâldedir
running	Süreç o anda işlemcide çalışmaktadır
waiting	Süreç bir olayı (ör. G/Ç bitişi) beklemektedir
terminated	Süreç tamamlanmıştır

Durumlar arasındaki geçişler şöyle anlatıldı:

- **new → ready:** Bir süreç yaratıldıktan sonra teknik olarak hazır durumdadır. İşlemcide çalıştırılabilir hâldeki bütün süreçler ready durumunda bulunur.
- **ready → running:** İşletim sistemi CPU'da yapacağı işi yaptıktan sonra, ready durumundaki süreçlerden birini seçip CPU'yu ona devretmelidir. Bu işe zamanlama (scheduling) denir ve nasıl yapıldığı ayrıca konuşulacaktır.
- **running → waiting:** Çalışan bir süreç bir olayı bekleyebilir. Bu olay örneğin G/Ç olabilir: süreç bir okuma başlatmıştır. İlk haftalarda konuşulan yapıya göre G/Ç denetleyicisine (I/O controller) bir iş verilir, iş bitince denetleyici haber verir. Arada DMA varsa denetleyici işini bitirir, DMA bilgiyi kopyalar ve sonra haber verilir. Bu süre boyunca sürecin bir şey yapmasına gerek yoktur ve CPU kaynağını başka bir süreç kullanmalıdır. İşin tamamlanmasını bekleyen süreç bekleme kuyruğunda (waiting queue) bekler.
- **→ terminated:** Süreç sisteme girer, bekler, çalışır, yeniden hazır olur, tekrar çalışır ve sonunda tamamlanıp sistemden çıkması beklenir. Tamamlanma normal de olabilir, anormal (abnormal) de. Her iki durumda da sürecin gideceği durum terminated'dır.

14.1 Terminated Durumuna Neden İhtiyaç Var?

Hoca bu soruyu özellikle sordu. Her süreç başka bir süreç tarafından yaratılır. Yaratan sürece ebeveyn (parent), yaratılan sürece çocuk (child) denir. Çocuk süreç belli bir görevi yerine getirmek için yaratılır.

Kabuk örneğinden gidersek: Bir komut yazarsınız. Komutun çalıştırılması için `fork` ile bir süreç yaratılır, `exec` ile üzerine kod yüklenir. Bu çalışmanın başarılı mı başarısız mı olduğunu, başına bir şey gelip gelmediğini takip etmek istersiniz.

Terminated durumu olmasaydı, biten süreç doğrudan ortadan kalkıp giderdi. O zaman ebeveyn sürecin, çocuğun nasıl bir sonuç ürettiği ya da nerede sorun yaşadığıyla ilgili bilgi alma şansı olmazdı. Bu yüzden görevini tamamlamış ve aslında sistemden çıkmış bir sürecin durumu takip edilebilsin diye böyle bir durum vardır.

> **Ek bilgi (derste anlatılmadı):** Unix'te bitmiş ama ebeveyni tarafından çıkış durumu henüz alınmamış sürece büyük olasılıkla "zombi süreç" (zombie process) denir. Ebeveyn `wait`/`waitpid` çağrısıyla çocuğun çıkış kodunu aldığında çocuğun kaydı sistemden tamamen silinir. Kabukta son komutun çıkış kodu `$?` değişkeniyle görülebilir.

15. Süreç Kontrol Bloğu (PCB)

Sistem her süreçle ilgili bir grup bilgi tutmalıdır. Bu bilgilerin tutulduğu yere süreç kontrol bloğu (PCB, Process Control Block) denir. Hoca bunun aslında bir yapı (structure) olduğunu, C'deki `struct` gibi düşünülmesini söyledi. Linux'taki tanımında adı `task_struct`'tır.

Alan adları ve içerik işletim sistemine göre farklılık gösterebilir, ama en basit hâliyle PCB'de şunlar bulunmalıdır:

- **Süreç kimliği:** Sistemdeki her süreci tanımlayan bir kimlik; genellikle süreç numarası (process number) ya da süreç kimliği (process ID, PID) olarak geçer.

- **Süreç durumu:** Sürecin hangi durumda olduğu.
- **Program sayacı:** Çalışmayı takip etmek için.
- **Yazmaçlar.**
- **Bellek yönetimi bilgisi:** Süreç bellekte hangi alanları kullanıyor?
- **G/Ç ve kaynak bilgisi:** Hangi dosyalara ve cihazlara erişiyor? Yani kaynak yönetimiyle ilgili kısımlar.
- **Hesap bilgisi (accounting):** İşletim sisteminin görevlerini sayarken hesap tutmaktan (accounting) ve durum bilgisinden söz edilmişti. Hesap için sürece verilen CPU zamanı gibi bilgiler sürekli takip edilmelidir. Bu takip PCB üzerinden yapılır.
- **CPU zamanlama bilgisi:** Bir sonraki başlıkta açıklanmıştır.

15.1 Zamanlama Bilgisi Neden PCB'de?

Zamanlama tekniklerinde farklı yaklaşımlar öngörülebilir:

- Çok statik bir yapı kullanabilirsiniz: Bir zaman dilimi (time quantum) boyunca her süreç çalıştırılır, bir sonraki zaman diliminde hazır kuyruğundan (ready queue) başka bir süreç alınır ve devam edilir. Round robin buna benzer bir yapıdır.
- Sistemde öncelikler (priority) tanımlayabilirsiniz: "Şu özelliğe sahip sürecin önceliği daha yüksek olsun" diyebilirsiniz.

Bu durumda zamanlamayla ilgili bütün bilgiyi bir yerde tutmanız gerekir. Özellikle öncelik geçmişe göre değişiyorsa bu daha da önemlidir. Örneğin sürecin son zaman dilimini kullanıp kullanmadığına, ne kadarını kullandığına bakıp "tamamını kullandıysa zaman dilimini artırayım" diyen bir yapı varsa, bu bilgiler de PCB'de tutulur.

16. Bağlam Değişimi

Hoca konuyu somutlaştırmak için iki süreçli bir sistem örneği verdi: P0 ve P1. Başlangıçta ikisi de çalışabilir durumdadır, yani ikisi de ready durumundadır.

1. P0 çalışmaya başlar.
2. P0 bir sistem çağrısı yapar. Örneğin kullanıcıdan klavye ile girdi alacaktır. Bunun için G/Ç aygıtına gidilir.
3. Girdi ne zaman girilip biteceği belli değildir. O zamana kadar P0'ı CPU'da tutmanın anlamı yoktur. CPU, çalışabilecek durumdaki başka bir sürece verilmelidir.
4. P0 sistem çağrısını yaptığı anda, P0'ın tekrar çalışabilmesini sağlayacak bütün bilgi (program sayacı, yazmaç kümesinin içeriği vb.) P0'ın PCB'sine kaydedilir ve P0 kenara alınır.
5. Kayıt bittikten sonra P1'in daha önce PCB'sine kaydedilmiş bütün bilgisi geri alınır: yazmaçlara ve gereken yerlere kopyalanır. Sonra PCB'deki program sayacı nereyi gösteriyorsa oradan çalışmaya devam edilir.
6. P1, bir kesmeye maruz kalana ya da bir sistem çağrısı yapana kadar çalışır. Kesmenin nedeni örneğin zamana dayalı zamanlama olabilir: sürece bir ya da iki zaman dilimi verilmiştir. Ayrılan süre dolunca zamanlayıcı kesmesi (timer interrupt) üretilir ve çalışma işletim sistemine geçer.
7. İşletim sistemi bu kesmenin zamanlamayla ilgili olduğunu görür ve süreci değiştirir. Önce P1'in güncel bilgisini P1'in PCB'sine yerleştirir.
8. Sistemde yalnızca iki süreç olduğu için P0'ın daha önce kaydedilmiş PCB bilgileri alınır, işlemciye ve belleğe yerleştirilir. P0 hayatına devam eder.

Slaytta bu işlem "CPU'nun süreçten sürece geçişi (CPU switch from process to process)" olarak anlatılır. Bu işlemin tamamına bağlam değişimi (context switch) denir.

16.1 İki Süreç de Beklerse

Soru: İki süreç de bekleme durumundaysa, yani birincisi G/Ç başlatıp bekleme kuyruğuna gitti, ikincisi de G/Ç başlatıp kuyruğa gitti ve ilkinin işi henüz bitmedi, o zaman CPU'da ne çalışır?

Cevap: Hoca soruyu önce kendi cümleleriyle yeniden kurarak doğru anladığını teyit etti. Böyle bir durumda örneğin Windows'ta "System Idle Process" çalışır; CPU boş kalmasın diye bu süreç çalıştırılır. Çünkü CPU'ya elektrik ve saat sinyali (clock) verdiğiniz sürece çalışır. Hoca

bunu musluğa benzetti: Açtığınız andan itibaren su akmaya başlar, siz kullansanız da kullanmasanız da.

17. İş Parçacıklarına Kısa Giriş

Şimdiye kadar tanımlanan sistemlerde sistemde birden fazla süreç vardır, ama her sürecin içinde tek bir akış vardır. Program (süreç) tek bir bellek konumundan başlar ve komutları adım adım yerine getirir. İş parçacıkları (threads) için ayrı bir bölüm vardır ve orada daha ayrıntılı konuşulacaktır.

Tek akışlı yapı şu anlamda etkin değildir. Bir süreç yaratmak için programın kodunu, datasını, heap'ini ve stack'ini kopyalarsınız. Benzer işi yapan birden fazla akışa ihtiyacınız varsa, her biri için bu yapının bir kopyasını daha oluşturmanız gerekir.

Bunu daha etkin yapmanın yolu iş parçacıklarıdır. Fikir şudur: Bütün data ve heap'i tekrar tekrar oluşturmak yerine, bu kaynakların çoğunu ortak kullanan ama birbirinden ayrı olan akışlar oluşturmak.

- Aynı kodu paylaşırlar.
- Aynı data bölümüne sahiptirler.
- Aynı heap'e sahiptirler.
- **Stack'leri ayrıdır.**

Stack'in neden ayrı olması gerektiği şöyle açıklandı: Bir akış bir fonksiyon çağıracağı zaman stack'e parametre basar. Farklı iş parçacıkları üst üste aynı stack'e basarsa ortalık birbirine girer; kimin nereye ne zaman ne bastığını takip edemezsiniz. Bu yüzden her iş parçacığı için ayrı stack yaratılır. Böylece birden fazla akış için süreç kullandığımızda harcayacağımız belleği daha etkin kullanmış oluruz.

Sistem çok iş parçacıklı (multi-threaded) bir yapıysa, PCB içinde bu iş parçacıklarıyla ilgili bilgiler de tutulmalıdır.

Hoca job, task ve process kavramlarının iç içe geçtiğini burada tekrar hatırlattı. Linux'ta C ile bir süreç yaratmak istediğinizde `fork` sistem çağrısını kullanırsınız. `fork` size bir süreç yaratır, ama hocanın ifadesiyle bu sürecin arka tarafta karşılığı olan bir iş parçacığı vardır ve bunun kontrol yapısının adı `task_struct`'tır. İsimlendirme bu yüzden iç içe geçmiştir.

> **Ek bilgi (derste anlatılmadı):** Linux çekirdeği büyük olasılıkla süreçleri ve iş parçacıklarını aynı `task_struct` yapısıyla temsil eder; çekirdeğin zamanladığı birim bu "task"tır. Bir süreçteki iş parçacıkları, bellek alanı gibi kaynakları paylaşan ayrı task'lar olarak yaratılır (`clone` çağrısı). Dersteki "fork bir süreç yaratıyor ama arkada bir thread var" ifadesinin anlamı budur.

18. Süreç Zamanlama Kuyrukları

Hoca, asıl derdin CPU kullanımını artırmak olduğunu hatırlattı; çünkü CPU değerli bir kaynaktır ve etkin kullanılmalıdır. Bunun için:

- G/Ç başlatan süreci kenara alıp CPU'yu başka sürece veririz.
- Zaman paylaşımı, yani çok görevlilik yaparız: Farklı görevler arasında geçiş sağlarız ki herkes CPU'dan istediği kaynağı biraz alabilsin, biraz ilerleyebilsin ve işler daha etkileşimli (interactive) olabilsin.

Süreç zamanlamasına (process scheduling) yüklediğimiz iş bu seçim sürecidir: Hazır durumdaki süreçlerden hangisinin bir sonraki CPU döngüsünde çalıştırılmaya başlanacağı. Zamanlama için farklı yöntemler ve algoritmalar vardır ve bunlar tek tek konuşulacaktır. Fakat hepsinde birkaç önemli veri yapısı ortaktır:

- **İş kuyruğu (job queue):** Sistemdeki bütün süreçlerin yer aldığı kuyruk.
- **Hazır kuyruğu (ready queue):** Bütün ihtiyaçları karşılanmış, bütün kaynakları almış ve tek eksiği işlemci olan, bir sonraki döngüde çalıştırılabilir hâldeki süreçler.
- **Aygıt (bekleme) kuyrukları (device queues):** Bekleme kuyruğu aslında alt kuyruklardan oluşmalıdır. Çünkü siz belli bir aygıtı beklersiniz. O yüzden genel bir bekleme

kuyruğunda değil, ilgili aygıtın bekleme kuyruğunda olmalısınız. Durumunuz "waiting" olsa da, hangi aygıtla ya da hangi yapıyla ilgili bekliyorsanız onun kuyruğundasınızdır.

Süreçler bu kuyruklar arasında geçiş yapar. Slaytta her aygıt için ayrı kuyruklar gösterilir. Hoca, veri yapıları dersini hatırlatarak bu kuyrukların bağlı liste (linked list) ile gerçeklendiğini söyledi. Böylece kuyruğa dinamik ve hızlı biçimde bir PCB ekleyip çıkarabilirsiniz.

18.1 Kuyruk Diyagramındaki Geçişler

- Hazır kuyruğundan CPU'ya geçerek running durumuna geçersiniz. Bütün kaynaklara sahipseniz hazır kuyruğundasınızdır. Diyagramda hazır kuyruğunun solunda ucu boşta duran ok, yeni yaratılan süreci gösterir: new durumundaki süreç, işlemci dışında bütün kaynaklara sahip olduğu için hayatına hazır kuyruğunda başlıyor gibi düşünülür.
- Zamanlayıcı sizi seçip bir sonraki döngü için işlemciyi size atarsa running durumundasınız.

Running durumundayken şunlar olabilir:

- **G/Ç isteği:** Okuma ya da yazma yönünde bir G/Ç başlatmak istemiş olabilirsiniz. Bu durumda başlattığınız G/Ç isteğine bağlı G/Ç kuyruğuna (bekleme kuyruğuna) gidersiniz.
- **Zaman diliminin bitmesi:** Zamanlama yapısına bağlı olarak size bir zaman dilimi verilmişse ve bu süre bittiyse bir kesme oluşur, CPU'dan alınırsınız ve hazır kuyruğuna dönersiniz.
- **Çocuk süreç yaratma:** `fork` da bir sistem çağrısıdır. Çocuk sürecin oluşturulması için kenara çekilirsiniz; işletim sistemi sizden bir süreç yaratır. Siz doğrudan hazır kuyruğuna dönersiniz, bir adım sonra çocuk süreç de ortaya çıkar. İkiniz birden hazır kuyruğundasınız. Bir sonraki adımda hanginizin çalışacağına CPU zamanlayıcısı karar verir. Hoca bunun bir örnek üzerinde ayrıca görüleceğini söyledi.
- **Kesme bekleme:** Süreç bir kesmeyi bekliyor olabilir.

  Hocanın vurgusu: Slayttaki kuyruk diyagramında CPU'dan "fork a child" kutusuna, oradan "child executes" kutusuna ve sonra hazır kuyruğuna giden yol kafa karıştırabilir. Bu, "ebeveyn fork dedi, çocuk tamamen çalışıp bitti, o sırada ebeveyn hep bekledi, sonra hazır kuyruğuna geldi" anlamına gelmez. Grafik bu anlamda biraz yanıltıcıdır. Fork sonrası ikisi de hazır kuyruğundadır ve hangisinin çalışacağına zamanlayıcı karar verir.

19. Kısa, Uzun ve Orta Vadeli Zamanlayıcılar

Zamanlama üç farklı isimle anılır; üç farklı zamanlayıcı vardır: kısa vadeli (short-term), uzun vadeli (long-term) ve orta vadeli (medium-term). Fakat "zamanlayıcı" dediğimizde hep kısa vadeli zamanlayıcıdan söz ederiz. Hoca bunun nedenini üçünü tanımladıktan sonra daha iyi anlayacağımızı söyledi.

19.1 Kısa Vadeli Zamanlayıcı

Kısa vadeli zamanlayıcı (CPU zamanlayıcısı), bir sonraki CPU döngüsünde CPU'nun hangi süreç tarafından kullanılacağını belirler. "Zamanlayıcı" denince aklımıza gelen kısım budur.

19.2 Uzun Vadeli Zamanlayıcı

Uzun vadeli zamanlayıcının (job scheduler) derdi şudur: Sistemde bir grup süreç vardır. Bunlardan hangileri ikincil bellekten (secondary storage) belleğe gelmeli ve kaçı belleğe konmalı?

Hoca, günümüzde kullandığımız işletim sistemlerinde doğrudan böyle bir yapı olmadığını söyledi. Çalıştırmak istediğiniz programlara bağlı olarak hangi süreçlerin oluşturulacağına kullanıcı ya da başka bir sistem karar verir. Bugünkü sistemlerde "kenara konmuş bir grup işi alıp hadi bunları çalıştır" gibi bir yapı yoktur.

Fakat uzun vadeli zamanlayıcı kavramı şunun için vardır: Kenardaki süreçler belleğe getirilirken aralarında bir **denge** olmalıdır. Bir süreç ya işlemcide çalışır ya da G/Ç isteği oluşturur; bu iki durum vardır. Süreç G/Ç'deyken CPU'yu kullanamayacağı için CPU'yu başka bir sürece

veririz. Çoklu programlamanın (multiprogramming) etkisi de CPU zamanını böyle optimize etmektir. Uzun vadeli zamanlayıcı, bunu optimize edecek şekilde süreç seçmekle ilgilidir. Belleğe süreç getirirken mümkün olduğunca hem G/Ç ağırlıklı (I/O bound) hem de CPU ağırlıklı (CPU bound) süreçler getirilmelidir.

- **Hepsi G/Ç ağırlıklı olursa:** Bir süreç biraz CPU'da çalışır, sonra G/Ç isteği oluşturduğu için bekleme kuyruğuna gider. Bir sonraki süreç çalıştırılır; o da CPU'yu azıcık kullanıp G/Ç kuyruğuna gider. Bir bakarsınız süreçler G/Ç kuyruğunda tren gibi, ip gibi dizilmiştir. Sonra G/Ç işini bitirenler hazır kuyruğuna gelir ve bu kez orada kuyruk oluştururlar. Sistem dengesiz olur.
- **Karışık olursa:** Bir kısmı G/Ç ağırlıklı, bir kısmı CPU ağırlıklı olursa sistem bir taraftan G/Ç'yi gerçekleştirirken bir taraftan da CPU ağırlıklı süreçleri çalıştırır. Daha çok iş üretilir.

19.3 Orta Vadeli Zamanlayıcı ve Takas

Hoca orta vadeli zamanlayıcıyı, diğer ikisi arasındaki orta yolu bulan yapı olarak tanımladı. Belleğe bazı süreçler getirdiniz, ama onlar şu an bir şekilde çalıştırılamıyor; örneğin fazla G/Ç beklemesi var. Bir taraftan da çalıştırmak istediğiniz başka süreçler var. Bir süreci bir kez belleğe koyduk diye ömür boyu orada kalması gerekmez.

Orta vadeli zamanlayıcı ile bu süreci bellekten çıkarabilirsiniz (swap out) ve yerine çalıştırmak istediğiniz diğer süreci belleğe alabilirsiniz (swap in). Bu işlemin genel adı takastır (swapping).

Sonuç olarak üç zamanlayıcı vardır, ama derste "zamanlama" denince hep kısa vadeli zamanlama konuşulacaktır ve sonraki konular bunun üzerine kurulacaktır. Ders slayt 3.18'de bitti.

19.4 Takasla Çıkarılan Süreç Nereye Gider?

Dersin sonunda bir öğrenci takas (swapping) ile ilgili bir soru sordu.

Soru: Takasta RAM'den çıkarılan süreç nereye gider ve nasıl geri gelir?

Cevap: Takasta amaç RAM'de bir alan açmaktır. RAM'de duran ama şu an çalıştırılmayan, büyük olasılıkla bir süre daha da çalışmayacak bir süreç çıkarılır. RAM'den sonraki basamak ikincil bellektir (secondary storage); süreç oraya çıkarılır. İkincil bellekten de bir süreç belleğe getirilir. Çıkarılan süreç ihtiyaç duyulduğunda yeniden RAM'e getirilir. Bunun nedeni şudur: Süreç, çalıştırılabilmek için ana bellekte (main memory) bulunması gereken bir yapıdır.

Örnek Sorular

Soru 1 (derste sözlü, 00:06:50): Slayttaki iki MS-DOS bellek resminde (solda sistem açılışı, sağda program çalışırken) sağda boş bellek daha azdır. İki resim arasında başka hangi fark vardır ve neden?

> **Çözüm (derste verildi):** Sağdaki resimde komut yorumlayıcısı daha küçüktür. Bu bir çizim hatası değildir. Program çalışırken komut yorumlayıcısının geçici (transient) kısmı bellekten kaldırılır, yalnızca yerleşik (resident) kısım kalır. Böylece programa daha fazla boş bellek kalır. Program bitince yerleşik kısım, geçici kısmı yeniden yükler.

Soru 2 (derste sözlü, 00:10:34): Unix'lerde bir süreç tamamlandığında izlenen genel teamül nedir?

> **Çözüm (derste verildi):** Süreç bir dönüş kodu üretir. Her şey normalse 0, bir hata ya da problem varsa 0'dan farklı bir değer döndürür. Kabuk, çalıştırdığı komutun başarılı olup olmadığını bu değerden anlar.

Soru 3 (derste sözlü, 00:27:25): Büyük (monolitik) bir çekirdeğin dezavantajı nedir?

> **Çözüm (derste verildi):** Çekirdek büyüdükçe ihtiyaç duyduğu bellek artar; oysa işletim sistemi ne kadar az kaynak kullanırsa o kadar iyidir. Ayrıca büyük çekirdeği yüklemek ve onunla diğer programları çalıştırmak sistemi yavaşlatır. Örneğin zamanlama kararı çok kısa sürede verilmelidir; yoksa CPU işi yapmaya değil, işi kimin yapacağına karar vermeye harcanır.

Soru 4 (derste sözlü, 00:55:34): Süreç durum diyagramında terminated durumuna neden ihtiyaç vardır?

> **Çözüm (derste verildi):** Her süreç bir ebeveyn süreç tarafından yaratılır. Ebeveyn, çocuğun başarılı mı başarısız mı bittiğini, bir sorun yaşayıp yaşamadığını öğrenmek ister. Biten süreç doğrudan ortadan kalksaydı bu bilgi kaybolurdu. Terminated durumu, görevini tamamlamış sürecin sonucunun takip edilebilmesini sağlar.

Soru 5 (derste sözlü, 01:03:02): İki süreçli bir sistemde iki süreç de G/Ç başlatıp bekleme kuyruğuna geçmişse CPU'da ne çalışır?

> **Çözüm (derste verildi):** CPU'ya güç ve saat sinyali verildiği sürece çalışır. Örneğin Windows'ta bu durumda CPU boş kalmasın diye "System Idle Process" çalışır.

Soru 6 (derste sözlü, 01:17:30): Uzun vadeli zamanlayıcı belleğe yalnızca G/Ç ağırlıklı süreçler getirirse ne olur?

> **Çözüm (derste verildi):** Her süreç CPU'yu kısa süre kullanıp G/Ç kuyruğuna gider; süreçler G/Ç kuyruğunda tren gibi dizilir, sonra hazır kuyruğunda yığılır. Sistem dengesiz olur. G/Ç ağırlıklı ve CPU ağırlıklı süreçler karışık getirilirse G/Ç ve CPU aynı anda kullanılır.

Soru 7 (derste sözlü, 01:21:19): Takasla RAM'den çıkarılan süreç nereye gider?

> **Çözüm (derste verildi):** İkincil belleğe (diske) çıkarılır. İhtiyaç duyulduğunda yeniden ana belleğe getirilir, çünkü süreç ancak ana bellekteyken çalıştırılabilir.

Dersin Toparlaması

Dersin ilk yarısında Bölüm 2'nin sonundaki işletim sistemi yapıları tamamlandı. Tek görevli ve basit MS-DOS'tan başlanıp çok kullanıcılı Unix'in monolitik çekirdeğine geçildi. Büyük çekirdeğin bellek ve hız sakıncaları, çekirdeği küçültme fikrini doğurdu: katmanlı yaklaşım, mesajlaşma maliyeti yüksek olan mikroçekirdek ve ihtiyaç anında yüklenip kaldırılan çekirdek modülleri. Bugünkü Linux, Windows ve Mac OS X'in bu yaklaşımları farklı oranlarda birleştiren karma yapılar olduğu vurgulandı.

İkinci yarıda Bölüm 3'e geçildi. Süreç, programın çalışan hâli olarak tanımlandı; bellekteki text, data, heap ve stack bölümleri, beş süreç durumu, her sürecin bilgisinin tutulduğu PCB ve bu bilginin kaydedilip yüklenmesiyle yapılan bağlam değişimi anlatıldı. İş parçacıklarına kısa bir giriş yapıldı, zamanlama kuyrukları ile kısa, uzun ve orta vadeli zamanlayıcılar tanıtıldı. Hoca, derste "zamanlama" denince hep kısa vadeli zamanlamanın konuşulacağını ve konunun bunun üzerinden devam edeceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.